

PDF output

Typeset every docs page as a PDF from the same document tree the web page comes from, with executed output included.

Shravan Goswami / <https://shravangoswami.com>

Almanac can typeset every docs page as a PDF during the build. Not a print stylesheet or a headless browser screenshot: a real typeset document, produced by [Typst](#) from the same tree the web page was rendered from.

This site has it on. Scroll to the bottom of any documentation page and the PDF link next to "Edit this page" gives you that page, typeset.

Executable code

is the interesting one: its PDF contains the same computed Python and R output the page shows.

That last part is the point. The PDF and the page cannot disagree, because there is one parse, one set of resolved cross references, and one set of executed results feeding both.

Turning it on

```
almanac({
  title: "My Project",
  future: { myst: true, pdf: true },
})
```

pdf requires myst, and the build says so if you forget:

```
[almanac] Invalid configuration:
- future.pdf: future.pdf needs future.myst: the PDF is built from the MyST tree
```

Install the toolchain, which is an optional peer like the rest:

```
npm install myst-to-typst @myriaddreamin/typst-ts-node-compiler
```

No system Typst installation is needed. The compiler is a native module with Typst built in.

What you get

One PDF per docs page, written to `dist/pdf/<page id>.pdf`, where the id is the same one the sidebar uses. So `docs/guides/writing.md` becomes `/pdf/guides/writing.pdf`.

Each page in the sidebar gains a PDF link next to "Edit this page". The link appears only when the feature is on.

What survives into the document:

- **Headings, lists, tables, and quotes**, typeset rather than screenshotted.
- **Math**, properly set. Inline and display both.
- **Numbered figures with captions**, using the same numbers the page shows.
- **Cross references**, with the text they resolved to.
- **Admonitions**, as coloured blocks.
- **Executed output**, printed under the code that produced it.

Deliberate limitations

- **Remote images are not embedded.** Typst cannot fetch over the network, and a build that reaches out to load a figure is a build that fails in CI behind a firewall. A remote image becomes a visible `[image: url]` line, and the build logs which page it was on. Local images are embedded normally.

- **A page that fails to typeset does not fail the build.** It is reported with the compiler's own diagnostics and skipped. The HTML for that page was already correct, and losing the whole site because one document confused the typesetter is the wrong trade.
- **Only docs pages.** Blog posts are not typeset.
- **Only the default version and locale.** A variant page can share its source file with the page it inherits from, so a variant PDF would silently be a copy of another one. The link is hidden there rather than pointing at something wrong.
- **Styling is minimal and not yet configurable.** A4, serif body, monospace code, sensible margins. Templates are the obvious next step and are not built.

Speed

The Typst compiler is created once per build and reused across pages, because it carries a font book and a parsed standard library that cost real time to build. A site of twenty pages typesets in a couple of seconds after that.

Nothing about the PDF step is cached between builds. Unlike code execution, where skipping unchanged work is the whole feature, typesetting is fast enough that a cache would mostly add ways to be wrong.